PYTHON KURSU • HAFTA 15 • GÜN 1

Makine Öğrenmesi 1: Kavramlar ve Regresyon

Ders 29 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Makine öğrenmesinin ne olduğunu ve gözetimli öğrenmenin türlerini açıklayabileceksiniz.
- scikit-learn'ün fit / predict kalıbıyla bir regresyon modeli eğitebileceksiniz.
- Veriyi eğitim ve teste ayırıp MAE, RMSE ve R^2 ile modeli değerlendirebileceksiniz.
- Sayısal ve kategorik özelliklerle çoklu regresyon kurabileceksiniz.
- Veri sızıntısını fark edip Pipeline ile tekrarlanabilir bir model hazırlayabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 28'in tekrarı
0:10–1:25	75 dk	Blok 1: ML nedir • öğrenme türleri • terimler • scikit-learn kalıbı • basit regresyon • tahmin • eğitim/test • MAE, RMSE, R^2
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: çoklu regresyon • katsayılar • kategorik özellikler • tahmin–gerçek grafiği • artıklar • veri sızıntısı • Pipeline • çapraz doğrulama • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 28'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Neden gerekli?

```
{% csrf_token %}
```

SORU 2 • Ne yapar?

```
form.save(commit=False)
```

SORU 3 • Ne olur?

```
@login_required
def panel(request): …
```

SORU 4 • Fark ne?

```
user.has_perm("blog.change_yazi")
yazi.yazar == request.user
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev: editör rolü ve yorum kuralları

```python
def clean_metin(self):
    metin = self.cleaned_data["metin"]
    if "http" in metin.lower():
        raise forms.ValidationError("Bağlantı içeremez.")
    if len(metin.split()) < 5:
        raise forms.ValidationError("En az 5 kelime yazın.")
    return metin
```

İYİ ÖRNEK

Editörler grubu

change_yazi + yorum izinleri + is_staff

SIK SORUN

izin verildi ama çalışmıyor

kullanıcı gruba eklenmemiş / oturum yenilenmemiş

BLOK 1 • 0:10–1:25 • 75 DK

Kavramlar ve ilk model

20 dk
ML nedir, türler, terimler

25 dk
scikit-learn ve basit regresyon

20 dk
Eğitim/test ve metrikler

10 dk
Hatalar

Makine öğrenmesi: kurallar yerine örnekler

Benzetme: deneyimli emlakçı

Yeni emlakçıya fiyat formülü vermezsiniz; yüzlerce satılmış evi gösterirsiniz. Bir süre sonra “Kadıköy’de 120 m², 10 yaşında bir 3+1 aşağı yukarı şu fiyata gider” der. Makine öğrenmesi bu deneyimi veriden çıkaran bilgisayar programıdır.

	Klasik programlama	**Makine öğrenmesi**
girdi	veri + kurallar	veri + doğru cevaplar
çıktı	cevaplar	kurallar (model)
örnek	KDV = fiyat × 0,20	ev fiyatı tahmini
ne zaman?	kural net ve basit	kural karmaşık / bilinmiyor

BLOK 1 • ML NEDIR, TÜRLER, TERIMLER • 20 DK

Öğrenme türleri

Tür	Veri	Görev	Örnek	Ders
Gözetimli – regresyon	doğru cevaplı	sayı tahmin et	ev fiyatı, sıcaklık	29, 30
Gözetimli – sınıflandırma	doğru cevaplı	sınıf tahmin et	spam mı? çiçek türü	30, 31
Gözetimsiz – kümeleme	cevapsız	benzerleri grupla	müşteri segmentleri	—
Zaman serisi	zamana bağlı	geleceği tahmin et	aylık satış, kur	32
Pekiştirmeli	deneme-ödül	strateji öğren	oyun oynayan ajan	—

Bugün: **gözetimli öğrenme • regresyon** — hedef bir **sayı** (fiyat).

Terimler: özellik, hedef, model

Terim	Anlamı	Bizim veride
örnek (satır)	bir gözlem	bir ilan
özellik (X)	girdi sütunları	metrekare, ilçe, oda, yaş…
hedef (y)	tahmin edilecek sütun	fiyat
model	X'ten y'ye giden kural	fiyat ≈ a × m² + b
eğitim (fit)	modelin veriden öğrenmesi	katsayıları bulmak
tahmin (predict)	yeni X için y	yeni ilanın fiyatı

KURAL

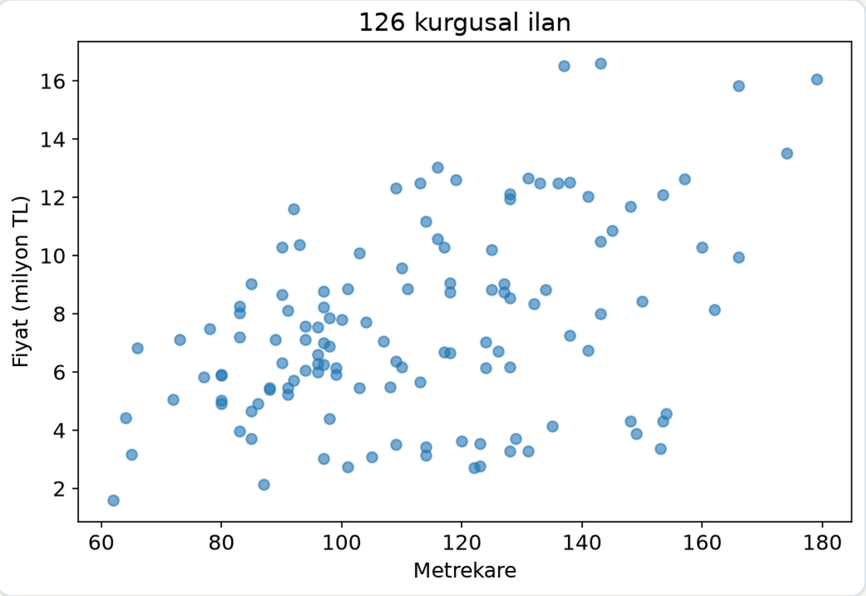
X büyük harf, y küçük

X bir tablo (çok sütun), y bir sütun — sklearn geleneği

Veri: Ders 23'ün temiz ilanları

```
print(df.shape)
sutunlar = ["ilce", "metrekare", "oda_sayisi", "bina_yasi", "fiyat"]
print(df[sutunlar].head(4).to_string(index=False))
print(df[["metrekare", "fiyat"]].corr().round(2))
```

```
ÇIKTI
(126, 11)
    ilce  metrekare  oda_sayisi  bina_yasi      fiyat
Beşiktaş      128.0           3         27 11940000.0
 Maltepe      108.0           2         34  5470000.0
 Maltepe       91.0           1         13  5220000.0
 Kadıköy      157.0           3         27 12620000.0
           metrekare  fiyat
metrekare       1.00   0.44
fiyat           0.44   1.00
```



BLOK 1 • SCIKIT-LEARN VE BASIT REGRESYON • 25 DK

scikit-learn kalıbı: oluştur → fit → predict

```
from sklearn.linear_model import LinearRegression

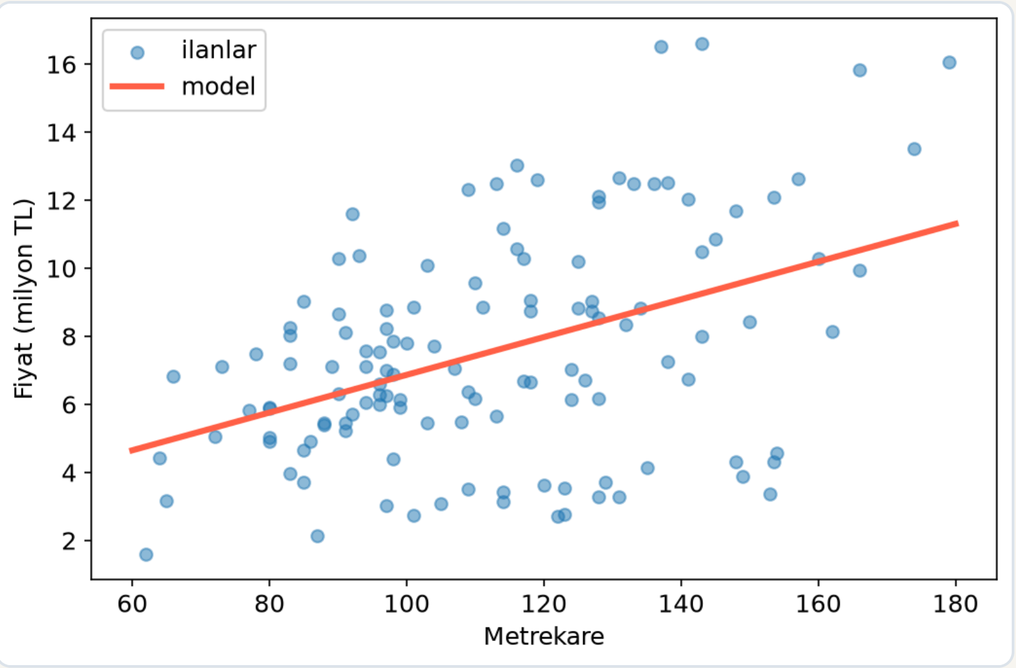
X = df[["metrekare"]]
y = df["fiyat"]
model = LinearRegression()
model.fit(X, y)
print("eğim:", round(model.coef_[0]), "TL / m²")
print("sabit:", round(model.intercept_), "TL")
```

```
ÇIKTI
eğim: 55454 TL / m²
sabit: 1322565 TL
```

Adım	Kod
1 • seç	`LinearRegression()`
2 • öğren	`model.fit(X, y)`
3 • tahmin	`model.predict(X_yeni)`

BLOK 1 • SCIKIT-LEARN VE BASIT REGRESYON • 25 DK

Regresyon doğrusu: verinin “ortalama eğilimi”




```
fiyat ≈ eğim × metrekare + sabit
```

ARTIK (hata)

gerçek – tahmin

noktanın doğruya dikey uzaklığı

DİKKAT

noktalar çok dağınık

metrekare tek başına fiyatı iyi açıklamıyor

predict: yeni evler için tahmin

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(df[["metrekare"]], df["fiyat"])
yeni = pd.DataFrame({"metrekare": [80, 120, 160]})
yeni["tahmin"] = model.predict(yeni[["metrekare"]]).round(-4)
print(yeni.to_string(index=False))
```

```
ÇIKTI
 metrekare     tahmin
        80  5760000.0
       120  7980000.0
       160 10200000.0
```

Tahmin her zaman **eğitim verisindeki örüntülere** dayanır.

400 m²'lik bir villa? Veride yok (en büyük 179 m²) → tahmin güvenilmez (**dışkestirim**).

BLOK 1 • EĞITIM/TEST VE METRIKLER • 20 DK

Eğitim ve test: modeli görmediği veriyle sınamak

Benzetme: sınav soruları

Öğrenci sınav sorularını önceden görüp ezberlediyse 100 alır ama konuyu öğrenmiş mi bilemezsiniz. Modeli de eğitimde görmediği ilanlarla sınarız.

```
from sklearn.model_selection import train_test_split

X_egitim, X_test, y_egitim, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)
```

Parça	Pay	Kullanım
eğitim (train)	%80 • 100 ilan	fit
test	%20 • 26 ilan	yalnızca değerlendirme
random_state	sabit sayı	her çalıştırmada aynı bölme

BLOK 1 • EĞITIM/TEST VE METRIKLER • 20 DK

Modeli ölçmek: MAE, RMSE, R²

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = df[["metrekare"]], df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = LinearRegression().fit(X_egitim, y_egitim)
tahmin = model.predict(X_test)
print(f"MAE : {mean_absolute_error(y_test, tahmin):,.0f} TL")
print(f"RMSE: {mean_squared_error(y_test, tahmin) ** 0.5:,.0f} TL")
print(f"R²  : {r2_score(y_test, tahmin):.2f}")
```

```
ÇIKTI
MAE : 1,870,155 TL
RMSE: 2,324,728 TL
R²  : 0.41
```

Ölçü	Anlamı	İyi değer
MAE	ortalama mutlak hata (TL)	küçük
RMSE	büyük hataları daha çok cezalandırır	küçük
R²	fiyattaki farkların ne kadarını açıklıyor	1'e yakın

BLOK 1 • SIK YAPILAN HATALAR

scikit-learn’de sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`model.fit(df["metrekare"], y)`	`ValueError: Expected a 2-dimensional container but got <class 'pandas.Series'> instead. Pass a DataFrame containing a single row (i.e. single sample) or a single column (i.e. single feature) instead.`	df[["metrekare"]]
`model.fit(df[["ilce"]], y)`	`ValueError: could not convert string to float: 'Beşiktaş'`	get_dummies (Blok 2)
`model.predict(…) # fit yok`	`sklearn.exceptions.NotFittedError: This LinearRegression instance is not fitted yet. Call 'fit' with appropriate arguments before using this estimator.`	önce fit

Hata vermeyen tuzak	Doğrusu
test verisiyle fit etmek	fit yalnızca X_egitim
eğitim başarısını rapor etmek	test (ya da çapraz doğrulama)
veri dışı aralıkta tahmin	dışkestirime dikkat

1:25–1:40 • 15 DK

Mola

Moladan sonra: daha çok özellik, ilçe bilgisi ve modeli doğru değerlendirmek

BLOK 2 • 1:40–2:55 • 75 DK

Daha iyi model, doğru değerlendirme

20 dk
Çoklu regresyon ve ilçe

15 dk
Grafiklerle değerlendirme

25 dk
Sızıntı, Pipeline, çapraz doğrulama

15 dk
Alıştırmalar

BLOK 2 • ÇOKLU REGRESYON VE ILÇE • 20 DK

Çoklu regresyon: birden çok özellik

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

ozellikler = ["metrekare", "oda_sayisi", "bina_yasi", "kat", "asansor"]
X, y = df[ozellikler], df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = LinearRegression().fit(X_egitim, y_egitim)
tahmin = model.predict(X_test)
print(f"eğitim R²: {model.score(X_egitim, y_egitim):.2f}")
print(f"test R²  : {r2_score(y_test, tahmin):.2f}  ·  MAE: {mean_absolute_error(y_test, tahmin):,.0f} TL")
```

```
ÇIKTI
eğitim R²: 0.27
test R²  : 0.30  ·  MAE: 1,964,825 TL
```

SÜRPRİZ

Testte daha kötü!

beş özellik, tek özellikten (R^2 0,41) daha iyi değil

BLOK 2 • ÇOKLU REGRESYON VE ILÇE • 20 DK

Yetersiz ve aşırı öğrenme

Durum	Eğitim başarısı	Test başarısı	Benzetme	Çare
yetersiz öğrenme	düşük	düşük	konuyu hiç çalışmamış	daha iyi özellik / model
uygun	yüksek	yükseğe yakın	konuyu anlamış	—
aşırı öğrenme	çok yüksek	düşük	soruları ezberlemiş	daha basit model, daha çok veri, düzenlileştirme

Az önceki 5 özellikli model (eğitim 0,27 • test 0,30): **yetersiz öğrenme** — önemli bilgi (ilçe) eksik.

BLOK 2 • ÇOKLU REGRESYON VE ILÇE • 20 DK

Kategorik özellik: ilçe (get_dummies)

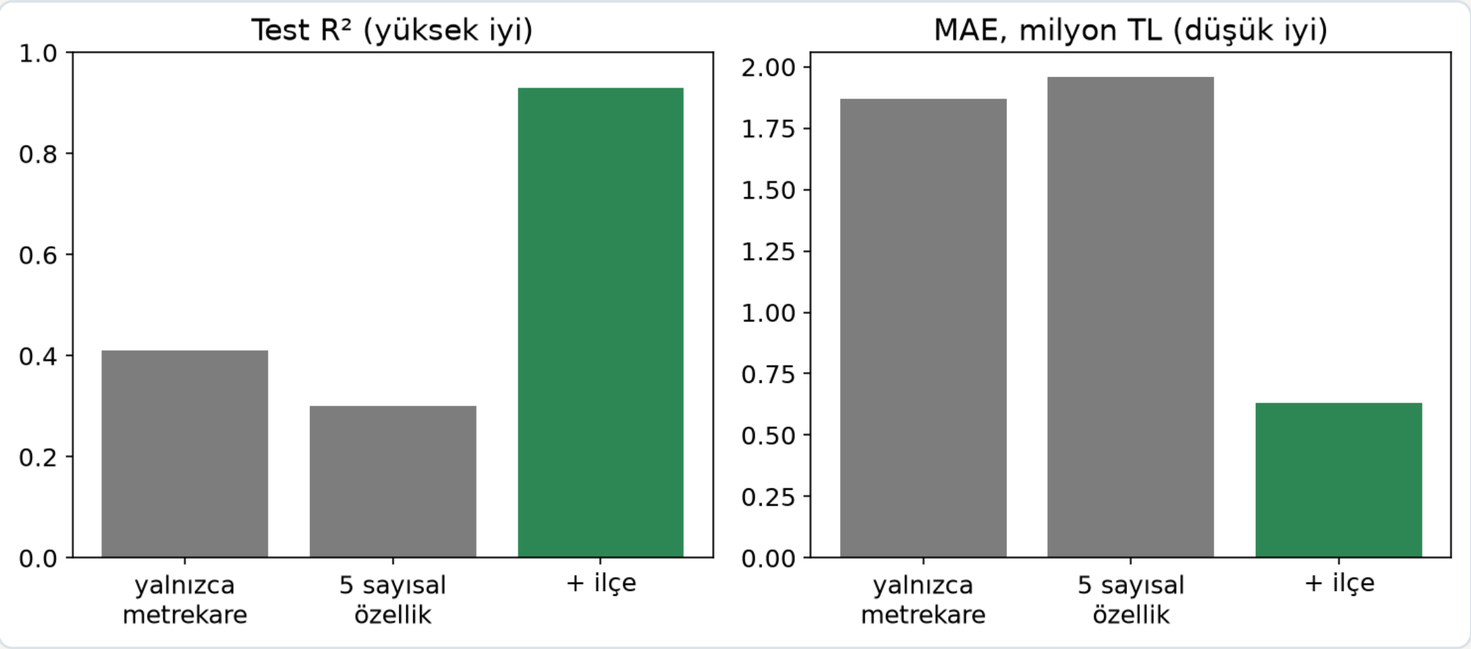
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

ozellikler = ["metrekare", "oda_sayisi", "bina_yasi", "kat", "asansor"]
X = pd.get_dummies(df[ozellikler + ["ilce"]], columns=["ilce"], drop_first=True, dtype=int)
y = df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = LinearRegression().fit(X_egitim, y_egitim)
tahmin = model.predict(X_test)
print(X.columns.tolist())
print(f"test R²: {r2_score(y_test, tahmin):.2f}  ·  MAE: {mean_absolute_error(y_test, tahmin):,.0f} TL")
```

```
ÇIKTI
['metrekare', 'oda_sayisi', 'bina_yasi', 'kat', 'asansor', 'ilce_Esenyurt', 'ilce_Kadıköy', 'ilce_Maltepe',
  'ilce_Üsküdar']
test R²: 0.93  ·  MAE: 634,267 TL
```

BLOK 2 • ÇOKLU REGRESYON VE ILÇE • 20 DK

Üç modelin karşılaştırması



Model	Test R²	MAE
yalnızca metrekare	0,41	1,87 mn
5 sayısal özellik	0,30	1,96 mn
+ ilçe	0,93	0,63 mn

BLOK 2 • ÇOKLU REGRESYON VE ILÇE • 20 DK

Katsayıları yorumlamak

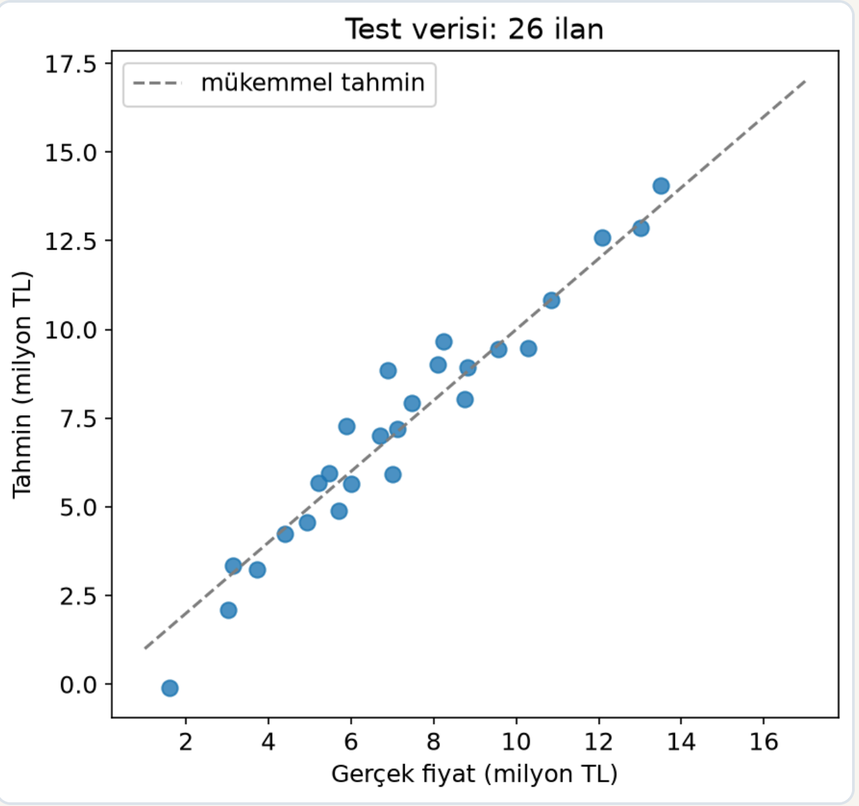
```
katsayi = pd.Series(model.coef_, index=X.columns).round(-3)
print(katsayi.to_string())
```

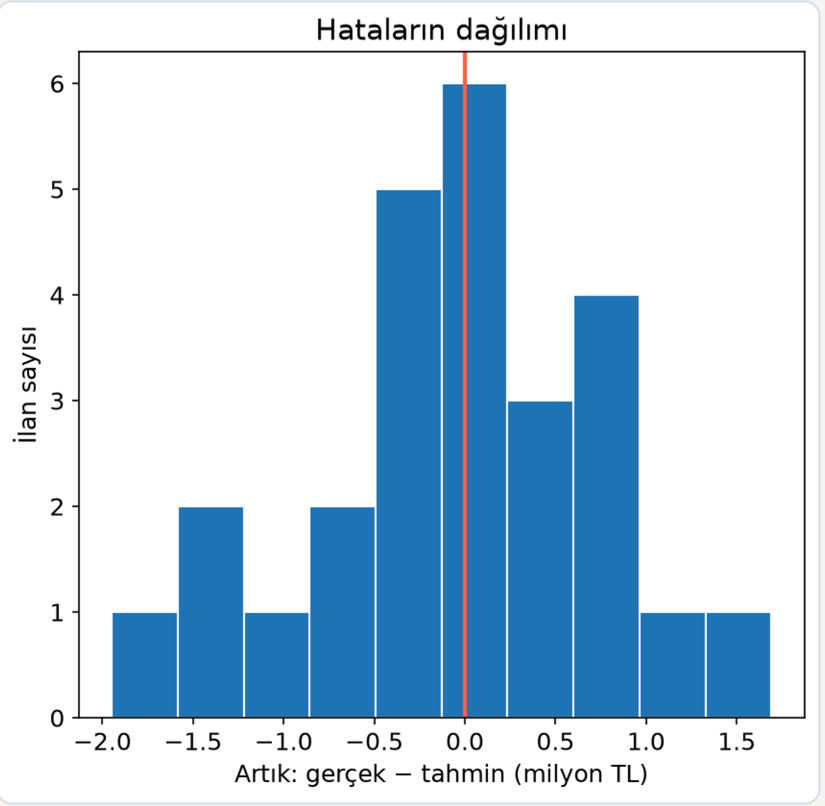
```
ÇIKTI
metrekare          74000.0
oda_sayisi       -184000.0
bina_yasi         -82000.0
kat                36000.0
asansor          -421000.0
ilce_Esenyurt   -8291000.0
ilce_Kadıköy    -1021000.0
ilce_Maltepe    -4346000.0
ilce_Üsküdar    -3233000.0
```

Özellik	Yorum
metrekare	+1 m² ≈ +74 bin TL
bina yaşı	+1 yıl ≈ −82 bin TL
Esenyurt	≈ −8,3 milyon (Beşiktaş'a göre)
Kadıköy	≈ −1,0 milyon

BLOK 2 • GRAFIKLERLE DEĞERLENDIRME • 15 DK

Grafiklerle değerlendirme: tahmin–gerçek ve artıklar





SOL

çizgiye yakın = iyi

noktalar köşegene yakın

SAĞ

artıklar 0 etrafında

sistematik sapma yok

BLOK 2 • SIZINTI, PIPELINE, ÇAPRAZ DOĞRULAMA • 25 DK

Veri sızıntısı: cevabı soruya gizlemek

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[["metrekare", "m2_fiyat"]]
y = df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = LinearRegression().fit(X_egitim, y_egitim)
print(f"test R²: {model.score(X_test, y_test):.3f}   ← şüpheli derecede iyi!")
```

```
ÇIKTI
test R²: 0.944   ← şüpheli derecede iyi!
```

> **Benzetme: kopya kâğıdı**
>
> m2_fiyat = fiyat ÷ metrekare. Modele m2_fiyat'ı vermek, sınavda cevap anahtarını sorunun altına yazmak gibidir. Yeni bir ilanda m2_fiyat'ı bilemeyiz — çünkü fiyatı bilmiyoruz!

Pipeline: hazırlık + model tek nesnede

```
# importlar: ColumnTransformer, OneHotEncoder, make_pipeline … (tamamı .py'de)
sayisal = ["metrekare", "oda_sayisi", "bina_yasi", "kat", "asansor"]
kategorik = ["ilce"]
hazirlik = ColumnTransformer([
    ("kategorik", OneHotEncoder(drop="first", handle_unknown="ignore"), kategorik),
    ("sayisal", "passthrough", sayisal),
])
pipe = make_pipeline(hazirlik, LinearRegression())

X, y = df[sayisal + kategorik], df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
pipe.fit(X_egitim, y_egitim)
print(f"test R²: {pipe.score(X_test, y_test):.2f}")
```

```
ÇIKTI
test R²: 0.93
```

Ham tablo girer → kodlama → model. **fit** ve **predict** tek çağrı; yeni ilan için aynı adımlar otomatik.

Çapraz doğrulama: tek bölmeye güvenmemek

```python
from sklearn.model_selection import cross_val_score

skorlar = cross_val_score(pipe, X, y, cv=5, scoring="r2")
print("5 katlı R²:", skorlar.round(2))
print(f"ortalama: {skorlar.mean():.2f} ± {skorlar.std():.2f}")
```

```
ÇIKTI
5 katlı R²: [0.88 0.9  0.93 0.9  0.94]
ortalama: 0.91 ± 0.02
```

Kat	Eğitim	Test
1	2·3·4·5	1
2	1·3·4·5	2
3	1·2·4·5	3
4	1·2·3·5	4
5	1·2·3·4	5

BLOK 2 • UYGULAMA

Uygulama: modeli kaydet ve yeni ilanları fiyatla

```
import joblib
import pandas as pd

pipe.fit(X, y)                                  # bütün veriyle son eğitim
joblib.dump(pipe, "ev_fiyat_modeli.joblib")

yuklenen = joblib.load("ev_fiyat_modeli.joblib")
yeni = pd.DataFrame([
    {"ilce": "Kadıköy", "metrekare": 110, "oda_sayisi": 3, "bina_yasi": 12, "kat": 4, "asansor": True},
    {"ilce": "Esenyurt", "metrekare": 110, "oda_sayisi": 3, "bina_yasi": 12, "kat": 4, "asansor": True},
])
yeni["tahmin"] = yuklenen.predict(yeni).round(-4)
print(yeni[["ilce", "metrekare", "tahmin"]].to_string(index=False))
```

```
ÇIKTI
    ilce  metrekare     tahmin
 Kadıköy        110 10210000.0
Esenyurt        110  3230000.0
```

BLOK 2 • SIK YAPILAN HATALAR

Modelleme hataları

Hatalı kod

```
model.predict(df[["metrekare",
"bina_yasi"]])
```

```
train_test_split(X, y.head(50))
```

Hata mesajı

```
- ...
```

```
ValueError: Found input variables with inconsistent
numbers of samples: [126, 50]
```

Çözüm

eğitimdeki sütunlar

X ve y aynı uzunlukta

Hata vermeyen tuzak

hedeften türetilmiş özellik (m2_fiyat)
tek bölmeye göre karar
get_dummies sütunları yeni veride farklı
fazla özellik = daha iyi sanmak

Doğrusu

tahmin anında bilinen özellikler
çapraz doğrulama
Pipeline + OneHotEncoder
test skoruyla karşılaştırın

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 15 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Bina yaşı

Yalnızca bina_yasi ile basit regresyon; katsayıyı yorumlayın; test R^2 ve MAE (random_state=1).

ORTA • 5 dk

2 • Hangi özellik?

Pipeline'a sırayla tek sayısal özellik + ilçe verin; 5 katlı çapraz doğrulamayla en iyi tek özelliği bulun.

ZOR • 6 dk

3 • En büyük hatalar

Test verisinde en çok yanılınan 5 ilanı ve ilçe bazında MAE'yi bulun; nedenini yorumlayın.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: bina yaşı

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

X, y = df[["bina_yasi"]], df["fiyat"]
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
model = LinearRegression().fit(X_egitim, y_egitim)
print(f"katsayı: {model.coef_[0]:,.0f} TL / yıl")
print(f"test R²: {model.score(X_test, y_test):.2f}")
print(f"MAE    : {mean_absolute_error(y_test, model.predict(X_test)):,.0f} TL")
```

```
ÇIKTI
katsayı: -97,132 TL / yıl
test R²: -0.12
MAE    : 2,443,639 TL
```

Çözüm 2: en iyi tek özellik (ilçe ile)

```
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder

sonuc = {}
for oz in ["metrekare", "oda_sayisi", "bina_yasi", "kat", "asansor"]:
    hazirlik = ColumnTransformer([
        ("ilce", OneHotEncoder(drop="first"), ["ilce"]),
        ("sayi", "passthrough", [oz])])
    pipe = make_pipeline(hazirlik, LinearRegression())
    X = df[["ilce", oz]]
    sonuc[oz] = cross_val_score(pipe, X, df["fiyat"], cv=5).mean()
print(pd.Series(sonuc).sort_values(ascending=False).round(2).to_string())
```

```
ÇIKTI
metrekare     0.83
oda_sayisi    0.74
bina_yasi     0.58
kat           0.52
asansor       0.51
```

Çözüm 3: en büyük hatalar ve ilçe bazında MAE

```
sonuc = X_test.copy()
sonuc["ilce"] = df.loc[X_test.index, "ilce"]
sonuc["gercek"] = y_test
sonuc["tahmin"] = tahmin.round(-4)
sonuc["hata"] = (sonuc["gercek"] - sonuc["tahmin"]).abs()
print(sonuc.nlargest(5, "hata")[["ilce", "metrekare", "gercek", "tahmin", "hata"]].to_string(index=False))
print(sonuc.groupby("ilce")["hata"].mean().round(-3).sort_values().to_string())
```

```
ÇIKTI
    ilce  metrekare    gercek    tahmin      hata
Beşiktaş        98.0 6890000.0 8840000.0 1950000.0
Esenyurt        62.0 1600000.0  -90000.0 1690000.0
 Kadıköy        97.0 8240000.0 9670000.0 1430000.0
 Kadıköy        80.0 5890000.0 7280000.0 1390000.0
 Üsküdar        97.0 7000000.0 5920000.0 1080000.0
ilce
Maltepe     385000.0
Üsküdar     508000.0
Beşiktaş    701000.0
Kadıköy     842000.0
Esenyurt    943000.0
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- ML = veriden kural öğrenmek; gözetimli: **regresyon** (sayı) ve **sınıflandırma** (sınıf).
- Kalıp: **model = …()** → **fit(X_eğitim, y_eğitim)** → **predict(X_test)**; X iki boyutlu.
- **train_test_split** + **MAE, RMSE, R²**; güvenilir ölçüm için **cross_val_score**.
- Doğru özellik (ilçe) özellik sayısından önemli; kategorikler **OneHotEncoder**.
- **Veri sızıntısına** dikkat; **Pipeline** + **joblib** ile tekrarlanabilir model.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kendi regresyonunuz

Açık veri setinizde (Ders 21–23) sayısal bir hedef seçin; en az 3 özellikle Pipeline kurun; test ve 5 katlı R^2, MAE raporlayın ve katsayıları yorumlayın.

2 • ZORUNLU

Tahmin–gerçek grafiği

Ödev 1'in modeli için tahmin–gerçek ve artık grafiklerini çizin; modelin en çok yanıldığı 3 örneği açıklayın.

3 • BONUS

Django'da fiyat tahmini

Kaydettiğiniz ev_fiyat_modeli.joblib'i Django'da bir form sayfasına bağlayın: ilçe, m², oda, yaş → tahmini fiyat.

Teslim: ders29_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 15 • GÜN 2

ML 2: düzenlileştirme ve sınıflandırma

Gelecek ders iki yeni araç:

- Ridge, Lasso, ElasticNet: aşırı öğrenmeye karşı
- StandardScaler ile ölçekleme
- Lojistik regresyon: sınıflandırma
- Doğruluk, karmaşıklık matrisi, precision/recall
- Türkçe yorumları olumlu/olumsuz sınıflandırma

Önizleme

```
from sklearn.linear_model import Ridge, LogisticRegression

ridge = Ridge(alpha=1.0)
siniflandirici = LogisticRegression()
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
scikit-learn başlangıç	scikit-learn.org/stable/getting_started.html
Doğrusal modeller	scikit-learn.org/stable/modules/linear_model.html
Çapraz doğrulama	scikit-learn.org/stable/modules/cross_validation.html
Pipeline ve ColumnTransformer	scikit-learn.org/stable/modules/compose.html